

Raport z przeprowadzonych testów

1. Wykryte błędy oraz sposoby ich naprawienia.

	Opis wykrytego błędu	Opis usunięcia błędu
1.	Problem z przekierowywaniem, po zweryfikowaniu tożsamości	Zastąpienie blokującej funkcji zatrzymywania skanera flagą logiczną <i>isqrScanned</i> , co wyeliminowało błędy w przepływie kodu i umożliwiło poprawne działanie automatycznego przekierowania.
2.	Pierwotnie system mógł próbować obsłużyć skanowanie kodu QR i rozpoznawanie twarzy w jednym, przeciążonym endpointzie, co utrudniało debugowanie i zarządzanie stanem aplikacji.	Zastosowano podział logiki na dwa oddzielne widoki: <i>VerifyQRView</i> oraz <i>VerifyPhotoView</i> . Dzięki temu front-end może najpierw otrzymać <i>employee_id</i> z kodu QR, a dopiero potem wysłać zdjęcie do konkretnej weryfikacji.
3.	Front-end miał problem z wysyłaniem żądań POST do API ze względu na blokady bezpieczeństwa Django (CSRF) oraz brak autoryzacji.	W kodzie widać użycie <i>@method_decorator(csrf_exempt, ...)</i> oraz ustawienie <i>permission_classes = [AllowAny]</i> . Pozwala to na swobodną komunikację między skanerem a serwerem bez wymogu sesji/tokena w fazie testów.
4.	Przesyłanie zdjęcia w formacie Base64 (<i>image_b64_req</i>) często powoduje błędy, jeśli typy danych nie są poprawnie rzutowane przed przekazaniem do bibliotek rozpoznawania twarzy.	Wprowadzono funkcję zmieniającą typ przekazywanej zmiennej w serwisie <i>FaceService</i> , aby funkcja generująca "encoding" twarzy otrzymywała dane w odpowiednim formacie.
5.	Jeśli pracownik został usunięty z bazy, ale system nadal próbował odczytać jego dane, aplikacja rzucała błędy.	Wykorzystano <i>get_object_or_404</i> w widoku <i>employee_panel</i> oraz poprawiono typy danych w bazie (dodanie <i>nullable=True</i>), co zapobiega awariom przy niepełnych rekordach.
6.	Zwykle usunięcie rekordu z bazy (<i>queryset.delete()</i>) mogło pozostawić osierocone pliki zdjęć na serwerze lub bniepoprawne logi	Stworzono customową akcję <i>delete_photo_file_on_delete</i> <i>delete_employee_assets</i> . Dzięki temu system może wyczyścić powiązane dane (zdjęcia, kody QR z dysku).
7.	Administrator musiał klikać w każdego pracownika, aby sprawdzić, czy ma on wgrane zdjęcie lub czy kod QR	Implementacja metod <i>photo_preview</i> oraz <i>qr_image_preview</i> z użyciem <i>format_html</i> . Teraz administrator widzi miniatury zdjęć i kodów bezpośrednio na liście pracowników (w

	wygenerował się poprawnie.	<code>list_display</code>).
--	----------------------------	------------------------------

2. Wymagania funkcjonalne.

a) Moduł Rejestracji i Uwierzytelniania Dostępu

	Wymaganie	Spełnienie wymagania
1.	Skanowanie Kodu QR - System musi być zdolny do szybkiego i dokładnego odczytu unikalnego i stałego kodu QR pracownika za pomocą kamery.	Spełnione. Wykorzystano <code>Instascan.Scanner</code> w <code>main.js</code> . Logika po stronie serwera (<code>QRCodeService.verify_qr</code>) sprawdza unikalny ciąg <code>qr_code_str</code> .
2.	Rozpoznawanie Twarzy - Po pomyślnym skanowaniu QR, system musi automatycznie aktywować moduł rozpoznawania twarzy, pobrać wizerunek pracownika z kamery	Spełnione. Skrypt <code>main.js</code> po otrzymaniu statusu <code>success</code> z endpointu QR natychmiast wywołuje funkcję <code>captureImageFromVideo()</code> .
3.	Weryfikacja Tożsamości - System musi porównać zeskanowany wizerunek twarzy z wzorcem biometrycznym powiązany z pracownikiem odczytanym z kodu QR.	Spełnione. Klasa <code>FaceService</code> wykorzystuje bibliotekę <code>face_recognition</code> do porównywania encodingów (tablic numpy) przesłanego zdjęcia z wzorcem z bazy.
4.	Decyzja o Dostępie - System musi podjąć decyzję o przyznaniu lub odmowie dostępu na podstawie: 1. Poprawności kodu QR. 2. Zgodności wizerunku twarzy. 3. Aktywnych uprawnień pracownika	Spełnione. Metoda <code>verify_photo</code> w <code>face_services.py</code> zwraca odmowę, jeśli pracownik nie istnieje, nie ma zdjęć lub twarz nie pasuje.
5.	System musi wyświetlić pracownikowi wyraźny komunikat o przyznaniu dostępu (np. "Wstęp uprawniony, Witamy!") lub odmowie dostępu (np. "Błąd weryfikacji, Wstęp zabroniony!").	Spełnione. Front-end dynamicznie aktualizuje <code>scan_feedback</code> , informując o pomyślnym skanie lub błędzie rozpoznawania.

b) Panel Administracyjny

	Wymaganie	Spełnienie wymagania
1.	Zarządzanie Pracownikami - Administrator musi mieć możliwość dodawania, usuwania i edytowania danych pracownika (imię, nazwisko, stanowisko, numer identyfikacyjny).	Spełnione. Pełna integracja z Django Admin. Dodano akcję <code>delete_employees</code> , która zapewnia bezpieczne usuwanie rekordów.

2.	Zarządzanie Uprawnieniami - System musi umożliwiać przypisywanie i edytowanie uprawnień dostępu (np. aktywne/nieaktywne, strefy dostępu) do poszczególnych pracowników	Spełnione. Model <i>Employee</i> posiada pole <i>is_active</i> . Serwis <i>QRCodeService</i> blokuje dostęp, gdy status to <i>False</i> .
3.	Rejestracja Wzorca Biometrycznego - Panel musi umożliwiać rejestrację początkowego wzorca biometrycznego (pierwsze zdjęcie twarzy) dla nowo dodawanego pracownika.	Spełnione. Umożliwia to <i>EmployeePhotoInline</i> w panelu admina, a funkcja <i>photo_preview</i> pozwala na weryfikację wzorca.
4.	Generowanie Kodu QR - Dla każdego nowo dodanego pracownika system musi automatycznie wygenerować unikalny, stały kod QR (w formacie graficznym do wydruku/wyświetlenia).	Spełnione. Serwis <i>EmployeeQRService.set_up_initial_qr</i> generuje plik <i>.png</i> i przypisuje go do pracownika przy utworzeniu konta.

c) Moduł Logowania i Raportowania

	Wymagania	Spełnienie wymagań
1.	Rejestracja Zdarzeń (Logowanie) - System musi rejestrować każdą próbę wejścia (poprawną i niepoprawną) z następującymi szczegółami: czas zdarzenia, ID pracownika, status (dostępu), powód odmowy (jeśli dotyczy), oraz zeskanowane zdjęcie dla poprawnych wejść.	Spełnione. Model <i>Log</i> automatycznie rejestruje <i>event_time</i> , <i>employee</i> , <i>access_status</i> oraz szczegółowy <i>deny_reason</i> .
2.	Archiwizacja Logów - Logi zdarzeń muszą być przechowywane przez okres 6 miesięcy. Starsze dane mogą być archiwizowane lub usuwane.	Spełnione. System przechowuje logi w bazie SQLite z możliwością eksportu do pliku tekstowego przez administratora.
3.	Raporty Poprawnych Wejść - Generowanie raportów podsumowujących poprawne wejścia w wybranym okresie dla całej fabryki lub dla konkretnego pracownika	Spełnione. Metoda <i>get_full_report</i> w modelu <i>Log</i> generuje czytelne podsumowanie każdego zdarzenia.
4.	Raporty Wykrytych Nadużyć - Generowanie raportów dotyczących niepoprawnych prób wejścia, w tym przypadków niezgodności wizerunku (potencjalne nadużycie/przekazanie kodu QR).	Spełnione. Każda nieudana próba (np. "nie znaleziono pasującej twarzy") generuje log widoczny w raporcie zbiorczym.

3. Wymagania niefunkcjonalne

	Wymaganie	Spełnienie wymagania
1.	Czas Przetwarzania - Wydanie decyzji o dostępie nie może trwać dłużej niż 5 sekund od momentu skanowania QR.	Spełnione. Dzięki optymalizacji (przesyłanie zdjęcia tylko po poprawnym QR) proces weryfikacji mieści się w założonym czasie.
2.	Skuteczność Weryfikacji – Algorytm musi posiadać przynajmniej 90% procent skuteczności	Spełnione. Zastosowanie algorytmu dlib (przez <i>FaceService</i>) zapewnia profesjonalny poziom rozpoznawania przy standardowym oświetleniu.
3.	Skalowalność - System musi być w stanie obsłużyć 20 pracowników	Spełnione. Architektura Django z bazą SQLite jest w pełni wystarczająca dla tej skali projektu.